

SimpleChat Documentation

CS 550 Advanced OS - Programming Assignment 3

Harun Pekacar - A20607262

Github Link: <https://github.com/0hr/SimpleChat/tree/assignment3>

Introduction

SimpleChat is a peer-to-peer (P2P) messaging application that demonstrates UDP-based, peer-to-peer messaging with anti-entropy. Instances discover neighbors on localhost, exchange vector clocks, and reliably deliver both direct and broadcast chat messages. It also supports Destination-Sequenced Distance-Vector (DSDV) routing, NAT traversal, and a no-forward rendezvous mode. It's built in C++ and the Qt6 framework.

This document provides an overview of the application's design, features, and building, implementation, and basic test cases.

Core Requirements

GUI Implementation

The application provides a basic graphical user interface (GUI) for users to interact with the chat system. The GUI is implemented using the Qt6 framework. GUI has the following requirements;

- Chat log shows received and sent messages with word wrap.
- A multi-line text input field allows users to compose and send messages.
- Destination picker filled with known peer IDs plus the broadcast value '-1'.
- A user can connect/disconnect/add a Peer.
- A user can send a private message to a peer by clicking the destination list. It will open the private message window.
- The routes table shows destination, next hop, sequence number, endpoint, and updated time.

SimpleChat : Id:2, port: 9002

Host: 27.0.0.1 ID: 2 My Port: 9002 Initial Peers: 1:9004
Connect Disconnect

Add Peer: 127.0.0.1 Port: 9002 Add

Routes

	Dest	Next Hop	SeqNo	Direct	Endpoint	Updated
1	3	3	4	Yes	127.0.0.1:9003	just now
2	4	4	4	Yes	127.0.0.1:9004	just now
3	2	1	4	No	127.0.0.1:9003	just now
4	1	1	4	Yes	127.0.0.1:9001	just now

Welcome!

UDP transport ready.

From 1 [#1]: Broadcast hi

Destinations

1
3
4

Broadcast

Type a broadcast message...

Send Broadcast

Routes Table

- **Dest:** destination node id
- **Next Hop:** next peer on the path
- **SeqNo:** freshness for routing, higher is newer
- **Direct:** Yes for neighbor, No for multi-hop
- **Endpoint:** last seen IP and port
- **Updated:** time since last update

Architecture Overview

UI Layer: SimpleChatMainWindow is the Qt Widgets interface. It has input, displays chat logs, and exposes buttons for connect/disconnect, sending, and adding peers.

Controller Layer: ChatController bridges UI signals to the transport (sendChat, requestPeer) and converts transport callbacks into UI updates.

Transport Layer: Core::UdpPeerTransport implements IChatTransport. It wraps QUdpSocket, manages peer discovery, maintains vector clocks, retransmits unacknowledged packets, and performs anti-entropy. It also manages DSDV routing, hop forwarding, and route rumors with NAT.

Destination-Sequenced Distance-Vector (DSDV)

- The initial route rumor is sent 0.5s after connecting to all neighbors. After that, a rumor is sent every 6,000 ms to one random neighbor.
- A rumor carries my routing sequence number, hop limit, and the last public IP:Port.
- When a rumor arrives, the receiver proposes/updates a route to the rumor's Origin via the sender. Newer SeqNo always wins. If it equals, a direct neighbor beats an indirect path.
- If the rumor advertises a reachable LastIP:LastPort, the receiver tries to add that endpoint as a peer.
- The rumor is forwarded to a random neighbor except the one it came from, with HopLimit decremented, so reachability information spreads.

Nat Traversal

- Each Route Rumor includes LastIP and LastPort (the sender's address/port as seen)
- On receive, the node saves that endpoint and tries to addPeer(LastIP, LastPort) to open a direct UDP path.
- When forwarding a rumor, the node updates LastIP/LastPort to the immediate sender's address/port.
- This helps peers behind NAT learn each other's reachable UDP tuples.

Rendezvous Mode (--noforward)

- Start the rendezvous node with --noforward.
- It still discovers peers, sends/forwards rumors, and exchanges summaries.
- It does not forward Chat messages from other peers; it only sends its own chat.
- Peers A and B connect to the rendezvous; Rumors carry LastIP/LastPort, so A and B learn each other, and each tries to addPeer.
- Once direct, A and B unicast to each other; the rendezvous keeps helping discovery/routing, but never relays their Chat.

Establishment of UDP P2P Network with DSDV, NAT Traversal, and No-Forwarding

In SimpleChat, the ring is established at startup based on command-line arguments:

- **--id:** Unique ID for this peer.
- **--port:** UDP port to listen on.
- **--peers:** Comma-separated seed peers as host:port.
- **--bind:** Local bind address (default 127.0.0.1).
- **--noforward:** Enable rendezvous mode; do not relay others' chats.
- **--test_message:** Message body to send in test mode.
- **--test_peer:** Destination peer ID for test messages.
- **--test_wait_time:** Delay before first test send (ms).
- **--test_count:** Number of test messages to send.

Build and Run the Project

A helper script, `build_run.sh`, is provided to build and run 4 nodes. They will be connected in a udp p2p with IDs 1, 2, 3, and 4 on ports 9001, 9002, 9003, and 9004.

1. **Make the script executable:** `chmod +x build_run.sh`
2. **Run the script:** `./build_run.sh`

Build Manually

1. **Create build directory:** `mkdir build`.
2. **Go into "build" directory:** `cd build`
3. **Generate build files:** `cmake ..`
4. **Compile the project:** `cmake --build.`

Run the project

To run an instance of the application, you need to specify the user ID, the port, and the peers.

```
./build/SimpleChat  
--id <id>  
--port <port>  
--peers <comma-separated-list-of-other-peer-ids>
```

For example, to start a chat client with ID "1" on port 9001, and there are 9002, 9003, 9004 peers, you would run:

```
./build/SimpleChat --id 1 --port 9001 --peers 127.0.0.1:9002,127.0.0.1:9003,127.0.0.1:9004
```

To create 4 peers, you would run the following commands in separate terminals:

```
./build/SimpleChat --id 1 --port 9001 --peers 127.0.0.1:9002,127.0.0.1:9003,127.0.0.1:9004
```

```
./build/SimpleChat --id 2 --port 9002 --peers 127.0.0.1:9001,127.0.0.1:9003,127.0.0.1:9004
```

```
./build/SimpleChat --id 3 --port 9003 --peers 127.0.0.1:9001,127.0.0.1:9002,127.0.0.1:9004
```

```
./build/SimpleChat --id 4 --port 9004 --peers 127.0.0.1:9001,127.0.0.1:9002,127.0.0.1:9003
```

Creating Linux Namespace

netns_setup.sh sets up Linux network namespaces to simulate NAT environments.

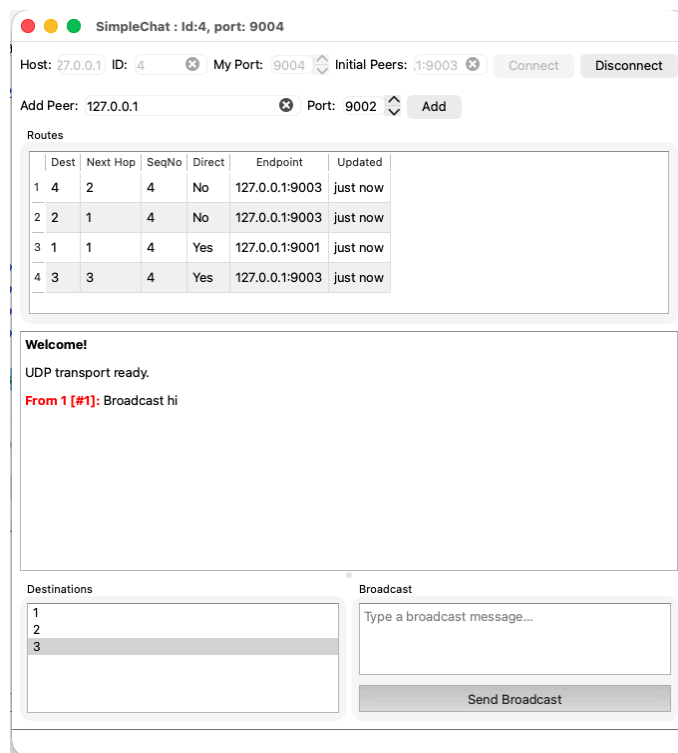
Netns_remove.sh removes the created NAT environments.

Basic Test Cases

Test Case 1

Test Case: Peer 1 sends a broadcast message to all peers

Expect: Peers 2, 3, and 4 each receive one broadcast from Peer 1



Test Case 2

Test Case: Peer 1 sends a private message to Peer 3 through Peer 2 after the routes form.

Expect: Peer 3 receives the message from Peer 1 via Peer 2

The screenshot shows the SimpleChat application window titled "SimpleChat : Id:3, port: 9003". At the top, there are input fields for "Host: 27.0.0.1", "ID: 3", "My Port: 9003", and "Initial Peers: 1:9002", along with "Connect" and "Disconnect" buttons. Below these is an "Add Peer" section with "Add Peer: 127.0.0.1" and "Port: 9002" and an "Add" button. A "Routes" table is displayed, showing two routes. Below the table, a "Welcome!" message is shown, followed by "UDP transport ready." and a received message: "From 1 [#1]: PM 1->3 via 2". At the bottom, there are two sections: "Destinations" with a list containing "1" and "2", and "Broadcast" with a text input field "Type a broadcast message..." and a "Send Broadcast" button.

	Dest	Next Hop	SeqNo	Direct	Endpoint	Updated
1	2	2	3	Yes	127.0.0.1:9002	just now
2	1	2	2	No	127.0.0.1:9001	just now

Welcome!
UDP transport ready.
From 1 [#1]: PM 1->3 via 2

Destinations

- 1
- 2

Broadcast

Type a broadcast message...

Send Broadcast

Test Case 3

Test Case: Peer 2 sends a direct private message to Peer 3.

Expect: Peer 3 receives exactly one direct message from Peer 2.

SimpleChat : Id:2, port: 9002

Host: 27.0.0.1 ID: 2

My Port: 9002

Initial Peers: 1:9004

Connect

Disconnect

Add Peer: 127.0.0.1

Port: 9002

Add

Routes

	Dest	Next Hop	SeqNo	Direct	Endpoint	Updated
1	2	3	4	No	127.0.0.1:9002	just now
2	4	4	4	Yes	127.0.0.1:9004	just now
3	1	1	4	Yes	127.0.0.1:9001	just now
4	3	3	4	Yes	127.0.0.1:9003	just now

Welcome!

UDP transport ready.

Me (2) → 3 [#1]: Hi 3

Destinations

1

3

4

Broadcast

Type a broadcast message...

Send Broadcast

Test Case 4

Test Case: Peer 1 sends a message to Peer 4 over two hops (1->2->3->4) using DSDV

Expect: Peer 4 receives the 1->4 message via the multi-hop DSDV route.

SimpleChat : Id:4, port: 9004

Host: 27.0.0.1 ID: 4 My Port: 9004 Initial Peers: 1:9003
Connect Disconnect

Add Peer: 127.0.0.1 Port: 9002 Add

Routes

	Dest	Next Hop	SeqNo	Direct	Endpoint	Updated
1	4	3	3	No	127.0.0.1:9004	just now
2	1	1	3	Yes	127.0.0.1:9001	just now
3	3	3	3	Yes	127.0.0.1:9003	just now
4	2	1	3	No	127.0.0.1:9002	just now

Welcome!

UDP transport ready.

From 1 [#1]: Hello 4 via 2,3

Destinations

1
2
3

Broadcast

Type a broadcast message...

Send Broadcast

Test Case 5

Test case: Two NAT peers (N1, N2) meet via rendezvous S (--noforward) and attempt direct messaging.

Expect: N1 and N2 learn endpoints via rumors, and N1->N2 direct message succeeds.

SimpleChat : Id:N2, port: 22222

Host: 0.0.1

ID: N2

My Port: 22222

Initial Peers: 5678

Connect

Disconnect

Add Peer: 127.0.0.1

Port: 9002

Add

Routes

	Dest	Next Hop	SeqNo	Direct	Endpoint	Updated
1	N1	N1	9	Yes	0.0.0.0:11111	just now
2	S	N1	7	No	10.200.0.1:45678	13s ago
3	N2	S	7	No	10.200.0.2:11111	11s ago

Welcome!

UDP transport ready.

From N1 [#1]: Hello N2 across rendezvous

Destinations

N1
S

Broadcast

Type a broadcast message...

Send Broadcast

Test Case 6

Test case: Peer 1 and Peer 3 connect via a rendezvous node 2 (--noforward), then Peer 1 sends to Peer 3.

Expect: Peers 1 and 3 discover each other via rumors; delivery may not occur through 2 due to no-forward.

SimpleChat : Id:3, port: 9003

Host: 27.0.0.1 ID: 3 My Port: 9003 Initial Peers: 1:9002 Connect Disconnect

Add Peer: 127.0.0.1 Port: 9002 Add

Routes

	Dest	Next Hop	SeqNo	Direct	Endpoint	Updated
1	2	2	3	Yes	127.0.0.1:9002	just now
2	1	2	2	No	127.0.0.1:9001	just now

Welcome!

UDP transport ready.

From 1 [#1]: PM to 3 via rendezvous

Destinations

1
2

Broadcast

Type a broadcast message...

Send Broadcast